

Flask-SQLAlchemy Lab 1 (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4v3-flask-sqlalchemy-lab-1>  <https://github.com/learn-co-curriculum/python-p4v3-flask-sqlalchemy-lab-1/issues/new>

Learning Goals

- Use Flask-SQLAlchemy to define a data model
 - Use SQLAlchemy-Serializer to serialize an object
 - Implement a Flask application that queries the database
 - Implement a Flask application that returns a JSON response
-

Setup

Fork and clone the lab repo.

Run `pipenv install` and `pipenv shell` .

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

Task #1: Define a model

Edit `server/models.py` to add a new model class named `Earthquake` that inherits from both `db.Model` and `SerializerMixin`.

Add the following attributes to the `Earthquake` model:

- a string named `__tablename__` assigned to the value `"earthquakes"`.
- a column named `id` to store an int that is the primary key.
- a column named `magnitude` to store a float.
- a column named `location` to store a string.
- A column named `year` to store an int.

Add a `__repr__` method to return a string that formats the attributes `id`, `magnitude`, `location`, and `year` as a comma-separated sequence as shown:

```
<Earthquake 1, 9.5, Chile, 1960>
```

Save `server/models.py`. Make sure you are in the `server` directory, then type the following to test the new `Earthquake` model class:

```
$ pytest testing/models_test.py
```

The 4 tests should pass. If not, update your `Earthquake` model to pass the tests before proceeding.

Task #2: Initialize the database

Now it is time to create a database named `app.db` with a table named `earthquakes`.

Execute the following commands within the `server` directory:

```
flask db init
flask db migrate -m "initial migration"
```

The `instance` and `migrations` folder should appear with the database file and a migration script.

Let's run the migration to create the `earthquakes` table:

```
flask db upgrade head
```

Take a look at the file `seed.py` :

```
#!/usr/bin/env python3
# server/seed.py

from app import app
from models import db, Earthquake

with app.app_context():

    # Delete all rows in the "earthquakes" table
    Earthquake.query.delete()

    # Add several Earthquake instances to the "earthquakes" table
    db.session.add(Earthquake(magnitude=9.5, location="Chile", year=1960))
    db.session.add(Earthquake(magnitude=9.2, location="Alaska", year=1964))
    db.session.add(Earthquake(magnitude=8.6, location="Alaska", year=1946))
    db.session.add(Earthquake(magnitude=8.5, location="Banda Sea", year=1934))
    db.session.add(Earthquake(magnitude=8.4, location="Chile", year=1922))

    # Commit the transaction
    db.session.commit()
```

Run the following command within the `server` directory to seed the table:

```
python seed.py
```

Use the Flask shell to confirm the 5 earthquakes have been added and id's have been assigned. Your output may differ depending on your implementation of the `__repr__()` method:

```
$ flask shell
>>> Earthquake.query.all()
[<Earthquake 1, 9.5, Chile, 1960>, <Earthquake 2, 9.2, Alaska, 1964>, <Earthquake 3, 8.6, Alaska, 1946>, <Earthquake
```

In the next step, you will implement views to query by id and filter by magnitude. But first you should practice some Flask-SQLAlchemy functions in the Flask shell.

Recall the `filter_by()` function selects rows having a specific value for a column. For example, to select the row matching a specific id `5` :

```
>>> Earthquake.query.filter_by(id=5).first()
<Earthquake 5, 8.4, Chile, 1922>
```

The `filter()` function selects rows matching a boolean expression. You can also use it to match a specific id:

```
>>> Earthquake.query.filter(Earthquake.id==5).first()
<Earthquake 5, 8.4, Chile, 1922>
```

Note that the `filter_by()` function can only test for equality. Use the `filter()` function if you need to use a different relational operator. For example, to get all quakes with a magnitude of at least 8.6, you need to use `filter()` with a boolean expression using the `>=` operator:

```
>>> Earthquake.query.filter(Earthquake.magnitude >= 8.6).all()
[<Earthquake 1, 9.5, Chile, 1960>, <Earthquake 2, 9.2, Alaska, 1964>, <Earthquake 3, 8.6, Alaska, 1946>]
```

Exit out of Flask shell and move on to the next task.

```
>>> exit()
```

Task #3: Add view to get an earthquake by id

Edit `app.py` to add a view that takes one parameter, an integer that represents an id. The route should have the form `/earthquakes/<int:id>`.

The view should query the database to get the earthquake with that id, and return a response containing the model attributes and values (id, location, magnitude, year) formatted as an JSON string. The response should include an error message if no row is found. Don't forget to import `Earthquake` from the `models` module.

For example, the URL <http://127.0.0.1:5555/earthquakes/2>  (<http://127.0.0.1:5555/earthquakes/2>) should result in a response with a 200 status and a body containing JSON formatted text as shown:

```
{
  "id": 2,
  "location": "Alaska",
  "magnitude": 9.2,
  "year": 1964
}
```

However, the URL <http://127.0.0.1:5555/earthquakes/9999>  (<http://127.0.0.1:5555/earthquakes/9999>) should result in a response with a 404 status and a body containing JSON formatted text as shown:

```
{
  "message": "Earthquake 9999 not found."
}
```

Test the route by typing the following within the `server` directory (make sure the Flask server is running):

```
pytest testing/app_earthquake_test.py
```

Make sure the 4 tests pass.

Task #4: Add view to get earthquakes matching a minimum magnitude value

Edit `app.py` to add a view that takes one parameter, a float that represents an magnitude. The route should have the form `/earthquakes/magnitude/<float:magnitude>` .

The view should query the database to get all earthquakes having a magnitude greater than or equal to the parameter value, and return a JSON response containing the count of matching rows along with a list containing the data for each row.

For example, the URL <http://127.0.0.1:5555/earthquakes/magnitude/9.0>  (<http://127.0.0.1:5555/earthquakes/magnitude/9.0>) should result in a response with a 200 status and a body containing JSON formatted text as shown:

```
{
  "count": 2,
  "quakes": [
    {
      "id": 1,
      "location": "Chile",
      "magnitude": 9.5,
      "year": 1960
    },
    {
      "id": 2,
      "location": "Alaska",
      "magnitude": 9.2,
      "year": 1964
    }
  ]
}
```

The URL <http://127.0.0.1:5555/earthquakes/magnitude/10.0>  (<http://127.0.0.1:5555/earthquakes/magnitude/10.0>) should result in a response with a 200 status and a body containing JSON formatted text as shown:

```
{
  "count": 0,
  "quakes": []
}
```

Test the route by typing the following within the `server` directory:

```
pytest testing/app_magnitude_test.py
```

Make sure the 3 tests pass.

Submit your solution

Save all files and rerun all tests:

```
pytest
```

Once all tests are passing, commit and push your work using `git` to submit.

This tool needs to be loaded in a new browser window

Load Flask-SQLAlchemy Lab 1 (CodeGrade) in a new window